



Instituto Infnet

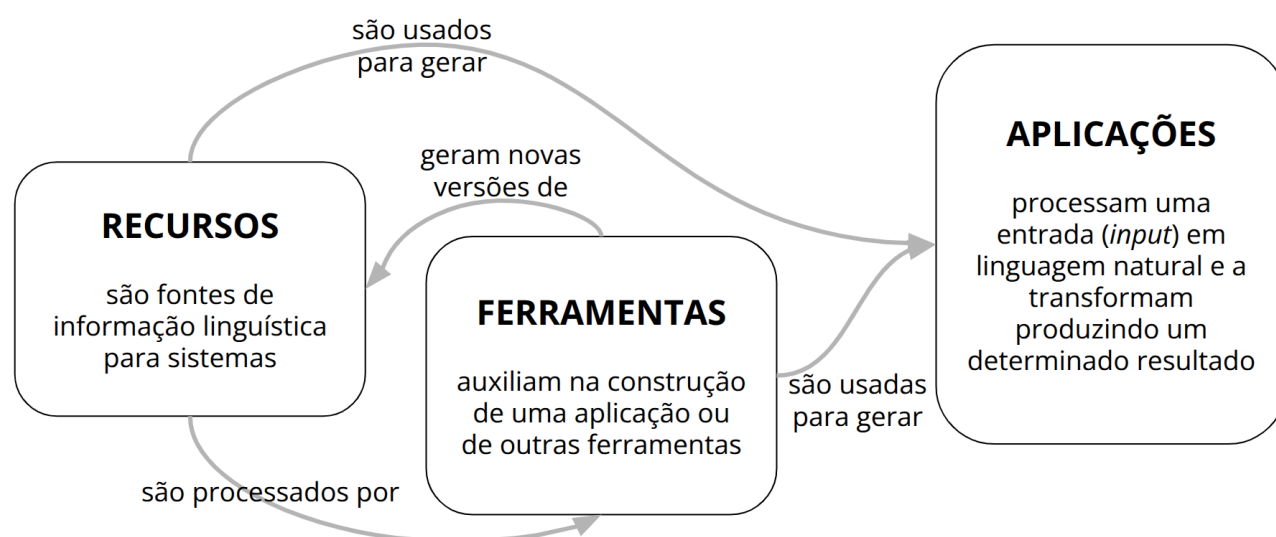
Bloco: 2º Ciclo - Fundamentos do Processamento de Dados

Python para Processamento de Dados [26E2_4]

Strings (str) - Continuação

Uma aplicação em Processamento de Linguagem Natural (PLN)

O Processamento de Linguagem Natural (PLN) é um campo de pesquisa que tem como objetivo investigar e propor métodos e sistemas de processamento computacional da linguagem humana. Na área da Ciência da Computação, PLN está ligado à área de Inteligência Artificial (IA) e também está intrinsecamente relacionada à Linguística Computacional [1].

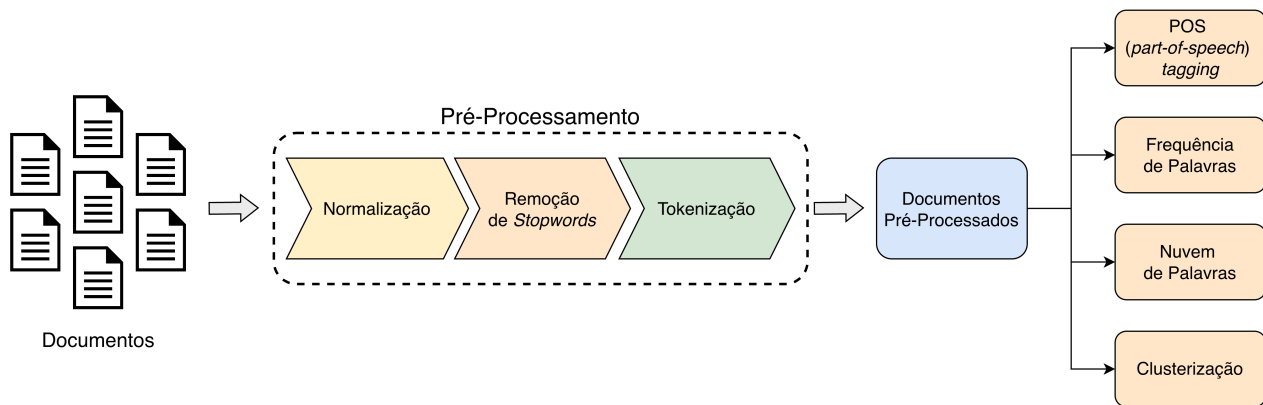


Para desenvolver praticamente qualquer aplicação de PLN, é necessário realizar fases/etapas que convencionamos chamar de pré-processamento. Nesse pré-processamento, algumas tarefas usuais são: segmentação do texto em sentenças (sentencição), separação de palavras (tokenização), tokenização em subpalavras (vetorização de subtokens), normalização de palavras (lematização e radicalização), entre outras [1].

Etapas de Pré processamento em PLN



Exemplo prático



```
texto_ata_copom = '''Atualização da conjuntura econômica do cenário do Copom ambiente externo se mantém adverso Apesar da  
print(texto_ata_copom)
```

```
Atualização da conjuntura econômica do cenário do Copom ambiente externo se mantém adverso Apesar da atenuação do estresse envolv
```

```
texto_normalizado_tokenizado_sem_stopwords, texto_preprocessado = pre_processamento(stopwords=stopwords_br, texto=texto_a  
print(texto_normalizado_tokenizado_sem_stopwords)  
print(texto_preprocessado)
```

```
['atualização', 'conjuntura', 'econômica', 'cenário', 'copom', 'ambiente', 'externo', 'mantém', 'adverso', 'atenuação', 'estresse  
atualização conjuntura econômica cenário copom ambiente externo mantém adverso atenuação estresse envolvendo bancos eua europa li
```

```

stopwords_br = [
    # Artigos e contrações
    'a', 'o', 'as', 'os', 'um', 'uma', 'uns', 'umas', 'ao', 'à', 'aos', 'às',
    'do', 'da', 'dos', 'das', 'no', 'na', 'nos', 'nas', 'pelo', 'pela', 'pelos', 'pelas',

    # Pronomes
    'eu', 'tu', 'ele', 'ela', 'nós', 'vós', 'eles', 'elas', 'me', 'mim', 'comigo',
    'te', 'ti', 'contigo', 'se', 'si', 'consigo', 'lhe', 'nos', 'vos', 'lhes',
    'meu', 'minha', 'meus', 'minhas', 'teu', 'tua', 'teus', 'tuas', 'seu', 'sua',
    'seus', 'suas', 'nosso', 'nossa', 'nossos', 'nossas', 'vosso', 'vossa', 'vossos', 'vossas',

    # Preposições
    'de', 'em', 'por', 'para', 'com', 'sem', 'sob', 'sobre', 'entre', 'até',
    'desde', 'contra', 'perante', 'através', 'além', 'dentro', 'fora', 'perto',
    'longe', 'durante',

    # Conjunções
    'e', 'mas', 'ou', 'porque', 'pois', 'que', 'se', 'como', 'quando', 'embora',
    'porém', 'todavia', 'contudo', 'então', 'também', 'apesar', 'caso', 'além disso',
    'portanto', 'logo', 'ainda que', 'a fim de', 'com o objetivo de',

    # Verbos auxiliares e comuns
    'ser', 'estar', 'ter', 'haver', 'ir', 'vir', 'fazer', 'poder', 'dever',
    'querer', 'saber', 'está', 'estão', 'tem', 'tenho', 'é',

    # Demonstrativos
    'esse', 'essa', 'isso', 'este', 'esta', 'isto', 'aquele', 'aquela', 'aquilo',
    'aqueles', 'aquelas', 'deste', 'desta', 'destes', 'destas', 'disso', 'daquilo',
    'nisto', 'naquilo',

    # Localizadores
    'lá', 'aqui', 'ali', 'onde', 'aonde'

    # Numericas
    'zero', '1', '2', '3', '4', '5', '6', '7', '8', '9', '0'
]

# Encapsulando o pre-processamento
def pre_processamento(stopwords, texto):
    """
    Definicao da funcao
    Etapas: Normalizacao, Tokenizacao (anagramas) e Remocao de Stopwords
    """
    texto = texto.lower() #Normalizacao
    tokens = texto.split() #Tokenizacao (anagramas)
    # list comprehension
    tokens = [token for token in tokens if token not in stopwords] # remocao das stopwords

    return tokens, ' '.join(tokens)

```

Contagem

```
# modo hard:
numero_ocorrencia_copom = 0
numero_ocorrencia_juros = 0
numero_ocorrencia_inflacao = 0
numero_ocorrencia_cenario = 0

for token in texto_normalizado_tokenizado_sem_stopwords:
    if token == 'copom':
        numero_ocorrencia_copom += 1

    if token == 'juros':
        numero_ocorrencia_juros += 1

    if token == 'inflação':
        numero_ocorrencia_inflacao += 1

    if token == 'cenário':
        numero_ocorrencia_cenario += 1

print(f'Numero de ocorrencia da palavra COPOM: {numero_ocorrencia_copom}')
print(f'Numero de ocorrencia da palavra JUROS: {numero_ocorrencia_juros}')
print(f'Numero de ocorrencia da palavra INFLAÇÃO: {numero_ocorrencia_inflacao}')
print(f'Numero de ocorrencia da palavra CENÁRIO: {numero_ocorrencia_cenario}')
```

```
Numero de ocorrencia da palavra COPOM: 12
Numero de ocorrencia da palavra JUROS: 8
Numero de ocorrencia da palavra INFLAÇÃO: 45
Numero de ocorrencia da palavra CENÁRIO: 9
```

```
print(f'Quantidade de caracteres do texto original: {len(texto_ata_copom)}')
print(f'Quantidade de caracteres do texto pre-processado: {len(texto_preprocessado)}')
print(f'Diferenca de caracteres do texto original e do texto pre-processado: {len(texto_ata_copom) - len(texto_preprocessado)}')
```

```
Quantidade de caracteres do texto original: 18517
Quantidade de caracteres do texto pre-processado: 15099
Diferenca de caracteres do texto original e do texto pre-processado: 3418
```

```
# Modo soft: usando o metodo count

ocorrencia_copom = texto_preprocessado.count('copom')

if numero_ocorrencia_copom == ocorrencia_copom:
    print(ocorrencia_copom)
```

```
12
```

```
palavras_para_analise = ['copom', 'juros', 'inflação', 'cenário', 'petróleo', 'preços']

for palavra in palavras_para_analise:
    ocorrencia_palavra = texto_preprocessado.count(palavra)
    print(f'Numero de ocorrencia da palavra {palavra.upper()}: {ocorrencia_palavra}')
```

```
Numero de ocorrencia da palavra COPOM: 12
Numero de ocorrencia da palavra JUROS: 8
Numero de ocorrencia da palavra INFLAÇÃO: 52
Numero de ocorrencia da palavra CENÁRIO: 12
Numero de ocorrencia da palavra PETRÓLEO: 1
Numero de ocorrencia da palavra PREÇOS: 13
```

```
# a importancia de normalizar o texto em NLP
```

```
print('Texto original')
ocorrencia_copom = texto_ata_copom.count('Copom')
print('Copom', ocorrencia_copom)
ocorrencia_copom = texto_ata_copom.count('COPOM')
print('COPOM', ocorrencia_copom)
ocorrencia_copom = texto_ata_copom.count('copom')
print('copom', ocorrencia_copom)

print('Texto Pre-processado')
ocorrencia_copom = texto_preprocessado.count('Copom')
print('Copom', ocorrencia_copom)
ocorrencia_copom = texto_preprocessado.count('COPOM')
print('COPOM', ocorrencia_copom)
ocorrencia_copom = texto_preprocessado.count('copom')
print('copom', ocorrencia_copom)
```

```
Texto original
Copom 12
COPOM 0
copom 0
Texto Pre-processado
Copom 0
COPOM 0
copom 12
```

Pesquisa de strings

```
# para pesquisar se uma string estah dentro de outro string, utiliza-se o metodo find
```

```
texto_preprocessado.find('copom')
```

```
41
```

```
texto_preprocessado[41]
```

```
'c'
```

```
texto_preprocessado[41:46]
```

```
'copom'
```

```
# find analisa da esquerda para a direita e retora um valor maior ou igual a zero, ou -1
texto_preprocessado.find('aonde')
```

```
-1
```

```
# se o objetivor for pesquisar da direita para a esquerda, utiliza-se o metodo rfind
texto_preprocessado.rfind('copom')
```

```
14902
```

```
texto_preprocessado[14902:14907]
```

```
'copom'
```

```
texto_preprocessado[41:]
```

```
'copom ambiente externo mantém adverso atenuação estresse envolvendo bancos eua europa limitado contágio condições financeiras mo
```

```
texto_preprocessado[41:].count('copom')
```

```
12
```

```
texto_preprocessado[14902:]
```

```
'copom analisaram evolução recente perspectivas economia brasileira economia internacional contexto regime política monetária cuj
```

```
texto_preprocessado[14902:].count('copom')
```

```
1
```

```
help(str.find)
```

Help on method_descriptor:

```
find(...) unbound builtins.str method  
S.find(sub[, start[, end]]) -> int
```

Return the lowest index in S where substring sub is found,
such that sub is contained within S[start:end]. Optional
arguments start and end are interpreted as in slice notation.

Return -1 on failure.

```
# pesquisa de todas as ocorrencias?
```

```
indice_ocorrencia = 0  
ocorrencia = 1  
while (indice_ocorrencia > -1):  
    indice_ocorrencia = texto_preprocessado.find('copom', indice_ocorrencia)  
    if indice_ocorrencia >= 0:  
        print(f'Ocorrencia {ocorrencia} da palavra COPOM na posicao : {indice_ocorrencia}')  
        ocorrencia += 1  
        indice_ocorrencia += 1
```

```
Ocorrencia 1 da palavra COPOM na posicao : 41  
Ocorrencia 2 da palavra COPOM na posicao : 2027  
Ocorrencia 3 da palavra COPOM na posicao : 6028  
Ocorrencia 4 da palavra COPOM na posicao : 7699  
Ocorrencia 5 da palavra COPOM na posicao : 8795  
Ocorrencia 6 da palavra COPOM na posicao : 11216  
Ocorrencia 7 da palavra COPOM na posicao : 11593  
Ocorrencia 8 da palavra COPOM na posicao : 12039  
Ocorrencia 9 da palavra COPOM na posicao : 13008  
Ocorrencia 10 da palavra COPOM na posicao : 13177  
Ocorrencia 11 da palavra COPOM na posicao : 13361  
Ocorrencia 12 da palavra COPOM na posicao : 14902
```

Conteúdo para aprofundamento

[Documentação oficial sobre strings](#)